
ADASPEC: ADAPTIVE KV-STATE-AWARE SPECULATIVE DECODING FOR HETEROGENEOUS BATCHES

Varun Donde¹ Justin Yo¹ Rafi Zaman¹

¹Carnegie Mellon University

ABSTRACT

The cost of verifying speculative tokens depends on the requests sharing a decode batch. We ask whether the mix of short and long contexts, and the associated KV-cache state, predicts verification time better than batch size and average context length alone. We will first measure this effect under controlled conditions on vLLM. If the added information improves prediction and draft-length decisions, we will use it to select how many tokens to draft at each step. We will compare throughput and latency with existing speculative decoding policies.

1 MOTIVATION AND HYPOTHESIS

In chain speculative decoding, a drafter proposes K successive tokens per request; the target verifies them together and commits an accepted prefix. Larger K can amortize a target pass over more output tokens, but increases draft and verification work. The best K therefore depends on both acceptance and verification cost.

Our hypothesis is that batches with equal size B , draft depth K , and mean context length can have different verification times when their *per-request* KV lengths differ. If so, a predictor using the batch’s context distribution and KV state may choose a better K than one using only B and mean length. The richer state is useful only if it improves decisions enough to offset its runtime cost.

2 RELATED WORK

L4 groups requests of similar lengths across serving instances because mixed lengths can make attention kernels inefficient (1). Vidur combines operator measurements with a simulator to predict decode and end-to-end serving performance (2). MagicDec shows that long KV caches change the benefit of speculative decoding even at large batch sizes, and uses a sparse-KV drafter; it studies long-context decoding rather than prefill scheduling (3).

vLLM can choose K from a user-provided batch-size table (4). SGLang changes draft steps using recent accepted lengths and batch size (5). vLLM’s DSpark adaptive verification uses a measured cost profile, but profiles one representative KV context length by default (6). Our question is whether the *live mix* of context lengths adds useful information to that cost estimate.

3 CONTROLLED EXPERIMENT

We will fix one open-source target/drafter pair, GPU type, attention backend, KV dtype, and engine version. Existing speculative-decoding configuration will set K ; a workload harness will control the requests. For fixed B and K , we will compare batches with the same mean context length but different mixtures: for example, eight 4k-token contexts versus four 1k and four 7k contexts. We will repeat this comparison across batch sizes, depths, and means.

We will record each request’s context length and allocated KV blocks, the verification token count, and target-pass time. GPU timings will distinguish attention-kernel time from the complete target pass; decode-step timing will include draft and scheduling overhead. We will warm up each shape, repeat runs, and hold out entire length mixtures for evaluation. Configuration alone cannot force a particular live KV distribution; the workload harness and instrumentation provide that control.

We will fit two simple predictors to the same measurements: $\hat{T}_{\text{base}} = f(B, K, \bar{L})$, $\hat{T}_{\text{KV}} = g(B, K, \bar{L}, L_{\text{max}}, H(L), N_{\text{blocks}})$. The first uses batch size, K , and average context length. The second also uses the longest context, counts of short and long requests ($H(L)$), and their KV-block count (N_{blocks}). We will compare errors on unseen batches and check whether either predictor chooses the same K as measured verification times would suggest. We will add the three extra inputs one at a time to learn which ones help.

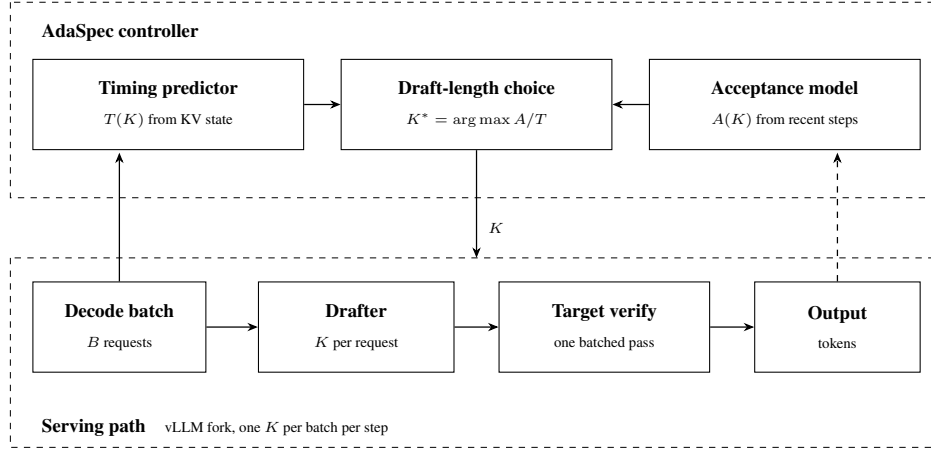


Figure 0. AdaSpec overview. A timing predictor (from the batch’s KV state) and an acceptance model (from recent steps) feed the draft-length choice of Eq. 1, which sets one K for the batch in the vLLM serving path. Dashed arrow: acceptance feedback from output tokens.

4 CHOOSING THE DRAFT LENGTH

For each possible K , we need two estimates: how many output tokens the batch will commit, and how long the draft-plus-verify step will take. We will start with recent acceptance measurements for the first estimate and our measured timing model for verification. Then we choose the K with the most expected output tokens per second:

$$K^* = \arg \max_K \frac{\hat{A}(K)}{\hat{T}_{\text{step}}(K)}. \quad (1)$$

Here $\hat{A}(K)$ is the expected number of output tokens committed by the batch, including any target-generated bonus tokens. $\hat{T}_{\text{step}}(K)$ is the predicted time to draft, verify, and schedule one step. Drafting farther helps only if its extra accepted tokens outweigh the extra time. We will first check choices offline against measured times, then implement the policy in a fork of vLLM. It will choose one K for the batch (Figure 0) and retain a fixed- K fallback. The timing predictor estimates *cost*; recent acceptance estimates *benefit*.

5 PERFORMANCE MEASUREMENTS

We will compare no speculation, fixed K , vLLM’s batch-size table, and our controller. We will also compare SGLang’s adaptive setting where the same model and draft method are supported. We will report output tokens/s, completed requests/s, p50/p95 inter-token latency, time to first token, accepted length, chosen K , and controller overhead. Offered-QPS-versus-throughput and offered-QPS-versus-latency plots will show the tradeoff under steady and changing arrival rates. We will test that output remains equivalent to the target’s decoding method and measure the cost of $K = 0$ rather than assuming it is free.

6 TEAM, MILESTONES, AND COMPUTE

Varun Donde, Justin Yo, and Rafi Zaman contributed equally to the initial system design and plan to divide the remaining work equally. Varun will lead the experimental design and isolate the effect of heterogeneous context lengths on vLLM speculative decoding at the Pittsburgh Supercomputing Center (PSC). Justin will lead predictor fitting and feature ablations. Rafi will lead the vLLM fork integration. All three will run and interpret the end-to-end evaluation.

By October 4, 2026, we aim to have the controlled experiment set up and an initial run completed. By October 25, we aim to have a trained verification-time predictor using the batch and KV features above. We will then integrate it into the vLLM fork and benchmark the controller.

We estimate 4–8 H100-80GB GPU-hours for single-GPU setup and pilot runs, followed by 16–24 hours on one eight-H100 node for distributed inference sweeps and repeats: 128–192 GPU-hours for that phase. Including the pilot gives roughly 132–200 GPU-hours, or 264–400 Bridges-2 service units at PSC’s H100 rate of two units per GPU-hour (7). This is a planning estimate, not an allocation request; the final number depends on the selected model, tensor-parallel size, and benchmark duration. PSC permits smaller GPU-shared jobs as well as full eight-GPU nodes (7).

REFERENCES

- [1] Yuan et al. L4: Low-Latency and Load-Balanced LLM Serving via Length-Aware Scheduling, 2025. <https://arxiv.org/abs/2512.19179>.
- [2] Agrawal et al. Vidur: A Large-Scale Simulation Framework for LLM Inference, MLSys 2024. <https://arxiv.org/abs/2405.05465>.

- [3] Chen et al. MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding, 2024. <https://arxiv.org/abs/2408.11049>.
- [4] vLLM Project. Dynamic Speculative Decoding. https://docs.vllm.ai/en/latest/features/speculative_decoding/dynamic_speculative_decoding/.
- [5] SGLang Project. Adaptive Speculative Decoding. https://docs.sglang.io/docs/advanced_features/adaptive_speculative_decoding.
- [6] vLLM Project. Adaptive Verification. https://docs.vllm.ai/en/latest/features/speculative_decoding/adaptive_verification/.
- [7] Pittsburgh Supercomputing Center. Bridges-2 User Guide. <https://www.psc.edu/resources/bridges-2/user-guide/>.